Discrete_Math

Mathematical_Equation
Truth_Tables (Zeros,Ones – True,False)
Code_Input – Code_Output
Using The Python Programming Language
Logic_Gates
Assignment_Truth_Tables (T,F)
* Laws
* Induction

Intended For :-
Dr\Sherien

#Class: [1]

Student Name :- Student Code :-

[Youssef Ahmed Alkamashany] 12500054

#(mathematical equation) :-

---

* $\neg [(A \oplus B) \rightarrow (C \wedge D)] \equiv (A \oplus B) \wedge (\neg C \vee \neg D)$

Part 1 (Left): $\neg [(A \oplus B) \rightarrow (C \wedge D)]$.

Part 2 (Right): $(A \oplus B) \wedge (\neg C \vee \neg D)$.

Final: Part 1 ≡ Part 2 .

Left Hand Side Steps:-

1. **Step 1:** Calculate $A \oplus B$ (XOR).
2. **Step 2:** Calculate $C \wedge D$ (AND).
3. **Step 3:** Calculate Step 1 -> Step 2 (Implication).
4. **Step 4: (LHS Final):** Calculate ¬ Step 3 (Negation).

Right Hand Side Steps:-

5. **Step 5:** Calculate $\neg C \vee \neg D$ (NOT C OR NOT D).
6. **Step 6: (RHS Final):** Calculate $(A \oplus B) \wedge$ Step 5 (AND).

Final Step: 7. **Equivalence Check:** Verify if LHS Final equals RHS Final.

"Truth table_(T,F)" :-

---

* $\neg [(A \oplus B) \rightarrow (C \wedge D)] \equiv (A \oplus B) \wedge (\neg C \vee \neg D)$

A	B	C	D	A⊕B	C∧D	(A⊕B)→(C∧D)	¬[(A⊕B)→(C∧D)]	¬C∨¬D	(A⊕B)∧(¬C∨¬D)	Equiv
F	F	F	F	F	F	T	F	T	F	T
F	F	F	T	F	F	T	F	T	F	T
F	F	T	F	F	F	T	F	T	F	T
F	F	T	T	F	T	T	F	F	F	T
F	T	F	F	T	F	F	T	T	T	T
F	T	F	T	T	F	F	T	T	T	T
F	T	T	F	T	F	F	T	T	T	T
F	T	T	T	T	T	T	F	F	F	T
T	F	F	F	T	F	F	T	T	T	T
T	F	F	T	T	F	F	T	T	T	T
T	F	T	F	T	F	F	T	T	T	T
T	F	T	T	T	T	T	F	F	F	T
T	T	F	F	F	F	T	F	T	F	T
T	T	F	T	F	F	T	F	T	F	T
T	T	T	F	F	F	T	F	T	F	T
T	T	T	T	F	T	T	F	F	F	T

“Truth table_(Zeros,Ones)” :-

---

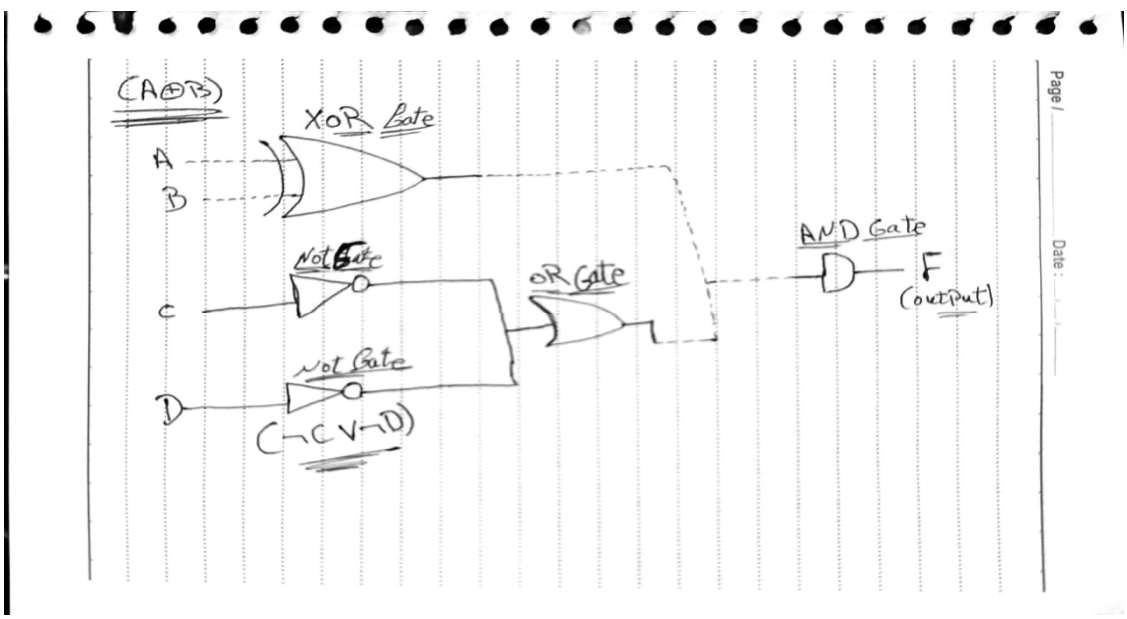
* ¬ [(A ⊕ B) → (C ∧ D)] ≡ (A ⊕ B) ∧ (¬C ∨ ¬D)

A	B	C	D	A⊕B	C∧D	(A⊕B)→(C∧D)	¬Imp (LHS)	¬C	¬D	¬C∨¬D	RHS Final	Equiv
0	0	0	0	0	0	1	0	1	1	1	0	1
0	0	0	1	0	0	1	0	1	0	1	0	1
0	0	1	0	0	0	1	0	0	1	1	0	1
0	0	1	1	0	1	1	0	0	0	0	0	1
0	1	0	0	1	0	0	1	1	1	1	1	1
0	1	0	1	1	0	0	1	1	0	1	1	1
0	1	1	0	1	0	0	1	0	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0	0	1
1	0	0	0	1	0	0	1	1	1	1	1	1
1	0	0	1	1	0	0	1	1	0	1	1	1
1	0	1	0	1	0	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0	0	0	0	0	1
1	1	0	0	0	0	1	0	1	1	1	0	1
1	1	0	1	0	0	1	0	1	0	1	0	1
1	1	1	0	0	0	1	0	0	1	1	0	1
1	1	1	1	0	1	1	0	0	0	0	0	1

Code Input :-

```python
import itertools

def to_tf(val):
    return "T" if val else "F"

def print_detailed_tf_proof():
    print("\n=DETAILED PROOF TABLE (T/F)")
    print("-" * 165)

    header = "| A | B | C | D | A⊕B | C∧D | (Imp)→ | LHS(¬) | ¬C | ¬D | ¬C∨¬D | RHS | Equiv |"
    print(header)
    print("-" * 165)

    combinations = list(itertools.product([0, 1], repeat=4))

    for combo in combinations:
        A, B, C, D = combo

        xor_ab = 1 if (A != B) else 0
        and_cd = 1 if (C and D) else 0

        if xor_ab == 1 and and_cd == 0:
            imp_res = 0
        else:
            imp_res = 1

        lhs_final = 1 if imp_res == 0 else 0

        not_c = 1 if C == 0 else 0
        not_d = 1 if D == 0 else 0

        not_c_or_not_d = 1 if (not_c or not_d) else 0

        rhs_final = 1 if (xor_ab and not_c_or_not_d) else 0

        match = 1 if (lhs_final == rhs_final) else 0

        print(f"| {to_tf(A)} | {to_tf(B)} | {to_tf(C)} | {to_tf(D)} | {to_tf(xor_ab)} | {to_tf(and_cd)} | {to_tf(imp_res)} | {to_tf(lhs_final)} | {to_tf(not_c)} | {to_tf(not_d)} | {to_tf(not_c_or_not_d)} | {to_tf(rhs_final)} | {to_tf(match)} |")

    print("-" * 165)

if __name__ == "__main__":
    print_detailed_tf_proof()
```

Code Output :-

```
DETAILED PROOF TABLE (T/F)
-----------------------------------------------------------------------------------------------------------------------
| A | B | C | D | A⊕B | C∧D | (Imp)→ | LHS(¬) | ¬C | ¬D | ¬C∨¬D | RHS | Equiv |
-----------------------------------------------------------------------------------------------------------------------
| F | F | F | F |  F  | F  |   T    |   F    | T  | T  |   T   | F   |   T   |
| F | F | F | T |  F  | F  |   T    |   F    | T  | F  |   T   | F   |   T   |
| F | F | T | F |  F  | F  |   T    |   F    | F  | T  |   T   | F   |   T   |
| F | F | T | T |  F  | T  |   T    |   F    | F  | F  |   F   | F   |   T   |
| F | T | F | F |  T  | F  |   F    |   T    | T  | T  |   T   | T   |   T   |
| F | T | F | T |  T  | F  |   F    |   T    | T  | F  |   T   | T   |   T   |
| F | T | T | F |  T  | F  |   F    |   T    | F  | T  |   T   | T   |   T   |
| F | T | T | T |  T  | T  |   T    |   F    | F  | F  |   F   | F   |   T   |
| T | F | F | F |  T  | F  |   F    |   T    | T  | T  |   T   | T   |   T   |
| T | F | F | T |  T  | F  |   F    |   T    | T  | F  |   T   | T   |   T   |
| T | F | T | F |  T  | F  |   F    |   T    | F  | T  |   T   | T   |   T   |
| T | F | T | T |  T  | T  |   T    |   F    | F  | F  |   F   | F   |   T   |
| T | T | F | F |  F  | F  |   T    |   F    | T  | T  |   T   | F   |   T   |
| T | T | F | T |  F  | F  |   T    |   F    | T  | F  |   T   | F   |   T   |
| T | T | T | F |  F  | F  |   T    |   F    | F  | T  |   T   | F   |   T   |
| T | T | T | T |  F  | T  |   T    |   F    | F  | F  |   F   | F   |   T   |
-----------------------------------------------------------------------------------------------------------------------

...Program finished with exit code 0
Press ENTER to exit console.
```

”Logic_Gates” :-